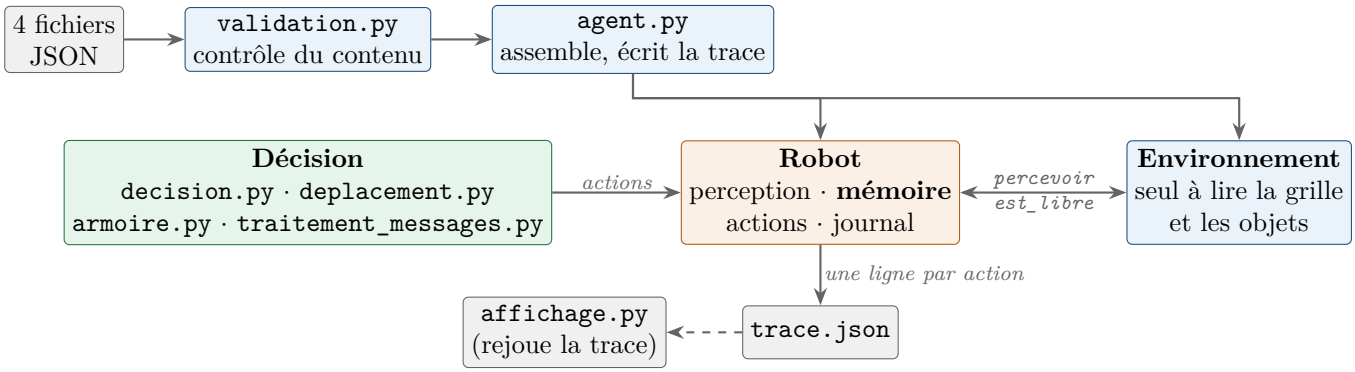
Projet 1 - Le robot de réconfort

Master 1 Informatique - POA - CAMARA Zéïnab Aly & HAJEE Neha
Enseignants : Vetea Jacot, Florian Lecourt - 2026-2027
Dépôt : `https://github.com/nehahajee/POA_Projet1` Vidéo : `demo.mp4`

1 Architecture



Couche	Où	Rôle
Perception	`Robot.percevoir`	lit les 4 voisines (mur, libre, armoire, dictionnaire, résident) et le casier courant devant l'armoire
Mémoire	`CarteMentale` (`robot.py`)	`{case: type}` ; une case absente est *inconnue*, supposée libre
Décision	`decision.py, deplacement.py, armoire.py, traitement_messages.py`	enchaîne une demande ; BFS ; 15 candidats triés (roue de Plutchik) ; premier mot reconnu
Action	`Robot.avancer, chercher, consulter, prendre, donner, attendre`	vérifie la légalité (`ValueError` sinon), met à jour l'état, journalise *avant* d'agir

Pourquoi ces frontières

- **Le robot ne reçoit jamais la grille** : `agent.py` lui passe la carte *sans* le champ `grille`. Il ne peut qu'interroger l'`Environnement` (`percevoir`, `est_libre`, `contenu_casier`), seul module autorisé à lire `grille` et `objet` (énoncé 3.2). La contrainte est donc tenue par la structure, pas par de la discipline.
- **La décision n'agit pas directement** : elle passe par les méthodes du robot, qui refusent une action illégale et journalisent. Un pas illégal est impossible à produire par accident.
- **La validation est séparée de l'agent** : elle lit la grille pour juger le *fichier* (22 cas de 5.6), mais ne dit jamais si un résident est atteignable.
- **Projet 2** : plusieurs `Robot` peuvent partager un même `Environnement` ; chaque robot garde sa propre mémoire.

2 Stratégie de replanification

Planifier sur une carte incomplète. Le robot cherche un chemin par *parcours en largeur* sur sa carte mentale, en supposant libre toute case inconnue (hypothèse optimiste, annexe B.5). L'armoire, le dictionnaire et les résidents ne sont pas traversables : la destination est l'ensemble de leurs cases adjacentes.
Quand un mur est découvert. Après chaque pas, le robot perçoit et met sa mémoire à jour. `deplacement()` vérifie, *avant* chaque pas, que la case suivante est encore traversable d'après la mémoire ; sinon il s'arrête avant de se cogner. `rejoindre()` **recalcule alors tout le chemin** depuis

la position courante : on ne répare pas le plan, car un BFS sur une grille de cette taille est bon marché et une réparation locale serait plus fragile. Différence avec l'annexe B.5 : on ne replanifie pas à chaque pas, *seulement quand le plan est invalidé.*

Mesure : `appartement_01`*, trajet vers R1 : le mur* $[6, 7]$ *est découvert en arrivant en* $[6, 6]$ *; 7 BFS, 31 pas, détour par le haut puis par* $[4, 11]$.

Coût. Un trajet coûte $1 + k$ BFS, où k est le nombre de plans invalidés ; chaque BFS est en $O(H \times L)$. L'algorithme termine : chaque replanification est causée par un mur *nouvellement* découvert, et il n'y en a qu'un nombre fini (un garde-fou lève une erreur si le même plan échoue deux fois).

Mesure : `scenario_01` *: 31 BFS pour 260 pas. Grand appartement* 24×40 *(*`test_grand`*) : 6 demandes, 1469 pas, 98 BFS, moins de 0,2 s (démarrage de Python compris).*

Quand aucun chemin n'existe. Les cases inconnues étant supposées libres, la carte mentale contient *au moins* tous les chemins réels : si le BFS échoue dessus, aucun chemin réel n'existe (annexe B.6). Le robot déclare l'échec sans jamais lire la grille.

Mesure : `test_dico_inaccessible` *: preuve en 11 pas pour la demande 1, puis* ***0 pas*** *pour la demande 2 : la mémoire s'en souvient.*

3 Critique de la règle de repli

La règle sur un exemple. Armoire dont la colonne *tristesse* est vide, message « Je me sens tellement seule ce soir » → tristesse/moyenne. Le casier exact est vide, ainsi que la colonne voisine *surprise* ; le robot retient donc *dégoût*/moyenne $[1, 5]$ (distance 1) et livre un **désodorisant** (`repli = voisine_1`).

Est-elle défendable ? *Pour* : elle est simple, déterministe, et vérifiable depuis la trace (deux binômes corrects livrent la même chose). *Contre* : la distance sur la roue de Plutchik mesure la *parenté* de deux émotions, pas la *compatibilité de réconfort*. Le même exemple le montre : dégoût est voisin de tristesse sur la roue, mais un désodorisant offert à une personne triste est déplacé, voire blessant. La règle préfère toujours *un objet* à *aucun objet*, alors qu'un mauvais objet peut être pire que rien. Enfin les deux voisines à distance 1 sont traitées de façon symétrique (le sens horaire ne sert qu'à départager), sans rien savoir de ce que l'objet *fait*.

Une meilleure règle. Remplacer la distance sur la roue par un *score de compatibilité émotion* × *objet* : chaque objet déclare les émotions qu'il apaise (avec un poids) et celles pour lesquelles il est à éviter. Le robot choisit l'objet de meilleur score, l'écart d'intensité servant de pénalité ; **sous un seuil, il ne donne rien** et échoue avec le motif `aucun objet adapté` plutôt que de livrer un objet inadapté.

Ce qu'il faudrait changer dans les fichiers. `armoire.json` : chaque casier gagne un champ `apaise` (liste d'objets `{emotion, poids}`) et un champ `evite` (liste d'émotions). Le rangement par position (colonne = émotion) devient une simple commodité, plus une règle. `armoire.py` : `calculer_candidats` trie par score au lieu de la clé 6.2. La **trace** : `repli` (4 valeurs fixées par `reconfort_io`) est remplacé ou complété par un champ `score`. `validation.py` : contrôle des nouveaux champs (émotions connues, poids dans $[0, 1]$).

4 Trois échecs de notre robot

Cette section présente trois situations où le robot se comporte de manière non optimale ou incorrecte, accompagnées d'une analyse de la cause et des solutions apportées ou envisagées.

4.1 Échec 1 : le résident muré bloquait tout le scénario

Dans cette situation, le robot prenait l'objet puis découvrait que le résident était inaccessible à cause des murs qui l'entouraient. En conséquence, le robot ne pouvait pas reposer l'objet, donc plus aucune action `PRENDRE` n'était possible et les demandes suivantes restaient bloquées. La cause vient de l'ordre des étapes : le robot ne découvrait l'inaccessibilité qu'après avoir pris l'objet, alors que l'énoncé ne prévoit aucune action pour le reposer et qu'il ne peut porter qu'un objet à la fois.
La correction mise en place consiste à vérifier l'accessibilité du résident avant `PRENDRE`, grâce à la perception et à la mémoire du robot, et jamais en lisant la grille. Si le résident est inaccessible, on déclare un échec « résident inaccessible » et on passe à la demande suivante, les mains vides. Sur `cartes_test/test_mure.json`, les demandes 2 et 3 (résident muré) échouent sans objet et la demande 4 réussit. Cette correction a un coût : le robot fait un aller-retour de plus vers le résident (sur `scenario_01`, le scénario passe ainsi de 160 à 260 pas). Il reste une limite pour une deuxième demande vers le même résident muré : le robot retourne quand même au dictionnaire et à l'armoire avant d'échouer (53 pas pour la demande 3 de `test_mure.json`), alors qu'un BFS sur sa carte mentale en début de demande suffirait à prouver l'échec en 0 pas.

4.2 Échec 2 : la négation, « je ne suis pas triste »

Lorsqu'un résident écrit « Je ne suis pas triste, je suis heureuse. », le robot comprend *tristesse/moyenne* et lui apporte une couverture, alors que le même résident qui écrit seulement « Je suis heureuse. » reçoit une photo de famille (*joie/moyenne*). Le robot apporte donc un objet de réconfort à quelqu'un qui va bien. Cet échec ne se voit pas dans la trace, la livraison étant notée comme réussie.
Ce comportement est normal au sens de l'énoncé : notre robot applique exactement la règle de la section 6.1, où le premier mot présent dans le dictionnaire donne la réponse. Ici, « triste » arrive avant « heureuse », et les mots « ne » et « pas » ne sont pas dans le dictionnaire, donc ils sont ignorés. La cause est structurelle : le robot lit les mots un par un, sans tenir compte du contexte sémantique de la négation.
Une solution consisterait à repérer les mots de négation (« ne », « pas », « jamais », « plus »). Si l'un d'eux précède un mot reconnu de près, on ignore ce mot et on continue la lecture, ce qui donnerait ici *joie*. Il faudrait pour cela ajouter une liste de négations dans `dictionnaire.json` et modifier `trouver_emotion` dans `traitement_messages.py`.

4.3 Échec 3 : l'impasse

Le robot ne perçoit que les 4 cases autour de lui et suppose libres les cases inconnues (hypothèse optimiste). Il peut donc s'engager dans un cul-de-sac avant de pouvoir le constater. Sur `cartes_test/test_impasse.json`, pour aller du départ au dictionnaire, il effectue 26 déplacements au lieu de 4. La route du sud est la plus courte, mais à égalité de longueur le BFS explore le nord en premier. Ensuite, chaque mur découvert n'allonge le nouveau plan que d'un ou deux pas : le robot remonte tout le couloir du nord, entre dans le renfoncement en `[2, 7]` et va jusqu'en `[1, 9]` avant de faire demi-tour. La demande réussit, mais elle coûte 74 pas. Ce n'est pas un bogue du BFS, mais le coût inhérent à l'hypothèse optimiste en univers inconnu.
Une solution serait de limiter l'exploration d'une direction inconnue : si le plan s'allonge de plus de quelques pas (par exemple 3) par rapport à l'estimation initiale, le robot essaie d'abord l'autre route. Une variante plus simple consiste à attribuer un coût supérieur aux cases inconnues comparativement aux cases déjà vues libres (recherche pondérée au lieu du BFS pur). Le robot privilégie alors les chemins qu'il connaît tout en conservant la capacité d'explorer.